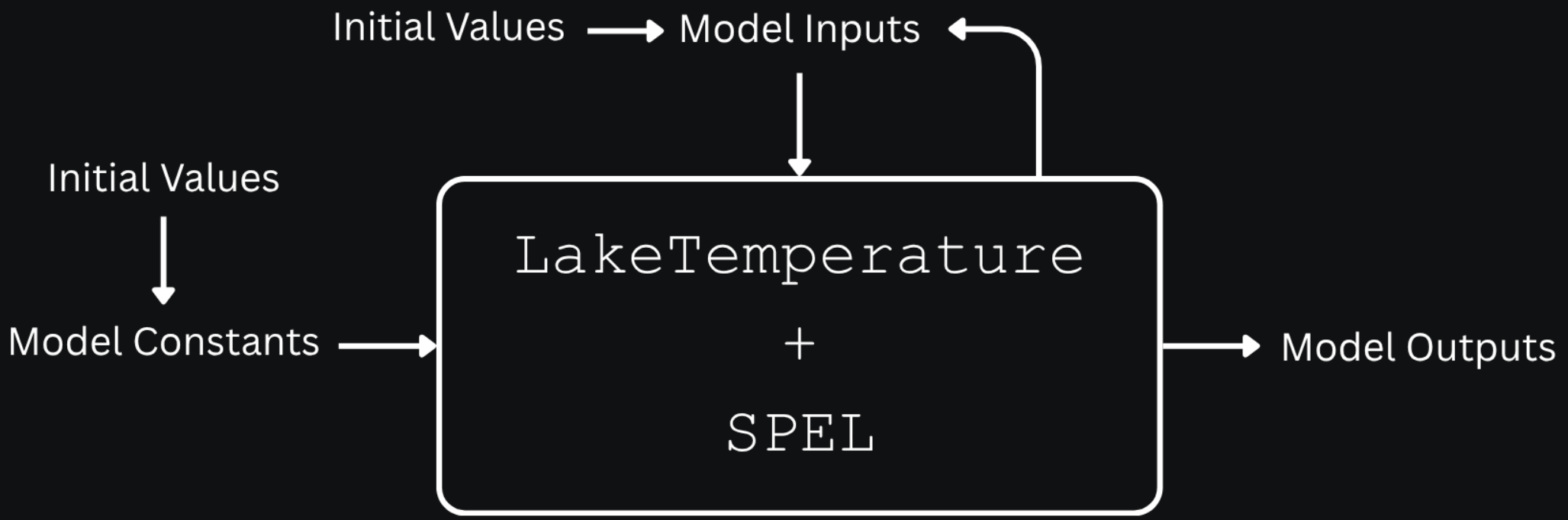
Change Impact Analysis & Combinatorial Testing

Slides by Sam Allen

The Testing Boundary



Part 1: Prelude: Change Detection Analysis

Change Detection Analysis

- Change detection analysis seeks to find how changes to individual model constants affect model outputs.
- It does this by changing a single scalar model constant across its range of values while holding all other model constants at their default values.
- Provides a simple form of qualitative analysis (one-constant-at-a-time) but misses interactions among model constants and their joint effect on output changes

Assumptions

- Initial analysis is built on a few key assumptions.
- First, not all the model constants have an effect on the model outputs.
- Second, model constants do not influence what values other model constants could take.
- Third, model constants' effects on model outputs aren't determined by preconditions being met.
- These assumptions underlie the preliminary results from the change detection analysis plugin.

Usefulness (Despite the Limitations)

- By discussing with the domain scientists and examining `LakeTemperature` source code, we found these assumptions do not always hold, meaning our initial results do not capture all of model outputs that the model constants affect.
- However, this doesn’t discount the results of our change detection analysis.
- It led to the discovery that certain model constants only affect model outputs under certain conditions.
- The results are still valid and useful when the model constants are using the default values provided by the domain scientists.

Rationale

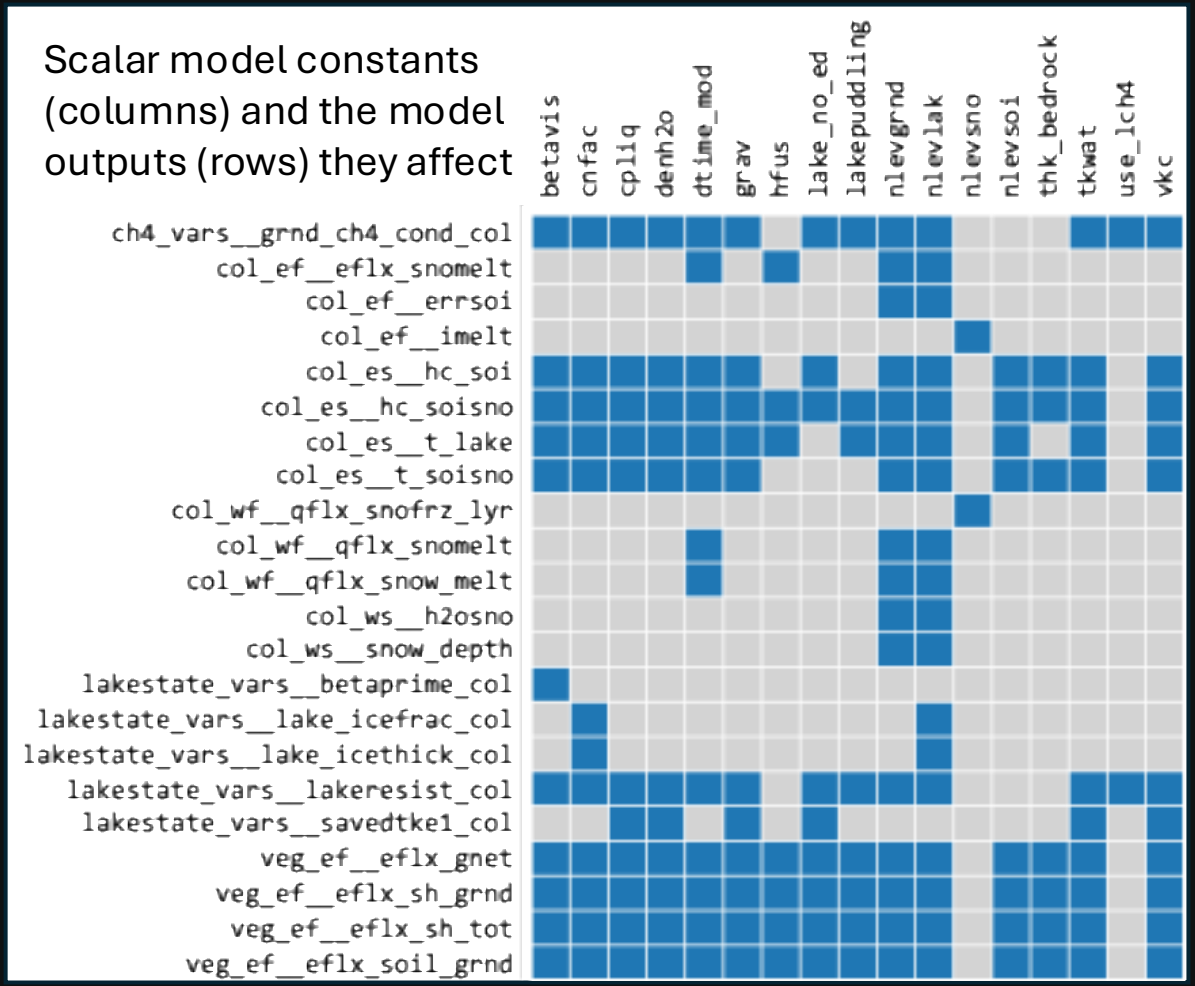
- To determine which model constants affect which model outputs, 11 cases were created for every scalar model constant, where all model constants were held at their default values except the one which was being tested, which was linearly interpolated to get 11 values evenly spread across its valid range.
- This assumed that model constants are independent of each other, and that no preconditions need to be met for a constant to have an effect on a model output.

Preliminary Results

Scalar model constants that had no effect on any model output

- cpice
- denice
- depthcrit
- iulog
- mixfact
- pudz
- tkair
- tkice

Scalar model constants (columns) and the model outputs (rows) they affect



Model outputs that didn't change in response to any model constant

- top_pp__active
- lun_pp__topounit
- lun_pp__itype
- lun_pp__lakpoi
- lun_pp__urbpoi
- lun_pp__active
- lakestate_vars__etal_col
- lakestate_vars__lake_raw_col
- lakestate_vars__ks_col
- lakestate_vars__ws_col
- soilstate_vars__watsat_col
- soilstate_vars__tkmg_col
- soilstate_vars__tkdry_col
- soilstate_vars__tksatu_col
- soilstate_vars__csol_col
- solarabs_vars__sabg_patch
- solarabs_vars__sabg_lyr_patch
- solarabs_vars__fsds_nir_d_patch
- solarabs_vars__fsds_nir_i_patch
- solarabs_vars__fsr_nir_d_patch
- solarabs_vars__fsr_nir_i_patch
- col_pp__gridcell
- col_pp__topounit
- col_pp__landunit
- col_pp__itype
- col_pp__active
- col_pp__snl
- col_pp__dz
- col_pp__z
- col_pp__zi
- col_pp__dz_lake
- col_pp__z_lake
- col_pp__lakedepth
- col_es__t_grnd
- col_ws__h2osoi_liq
- col_ws__h2osoi_ice
- col_ws__frac_iceold
- col_wf__qflx_snofrz
- veg_pp__topounit
- veg_pp__landunit
- veg_pp__column
- veg_pp__itype
- veg_pp__active

Limitations and Next Steps

- So far, the combinatorial test cases were only using the scalar model constants assumed to have an impact on model outputs. Next, they will include all scalar model constants.
- The change detection analysis might be redone in the future by examining preconditions through `LakeTemperature` source code, however, further combinatorial testing and change impact analysis will not depend on these results, as all scalar model constants will be included.

Part 2: Change Impact Analysis

Change Impact Analysis

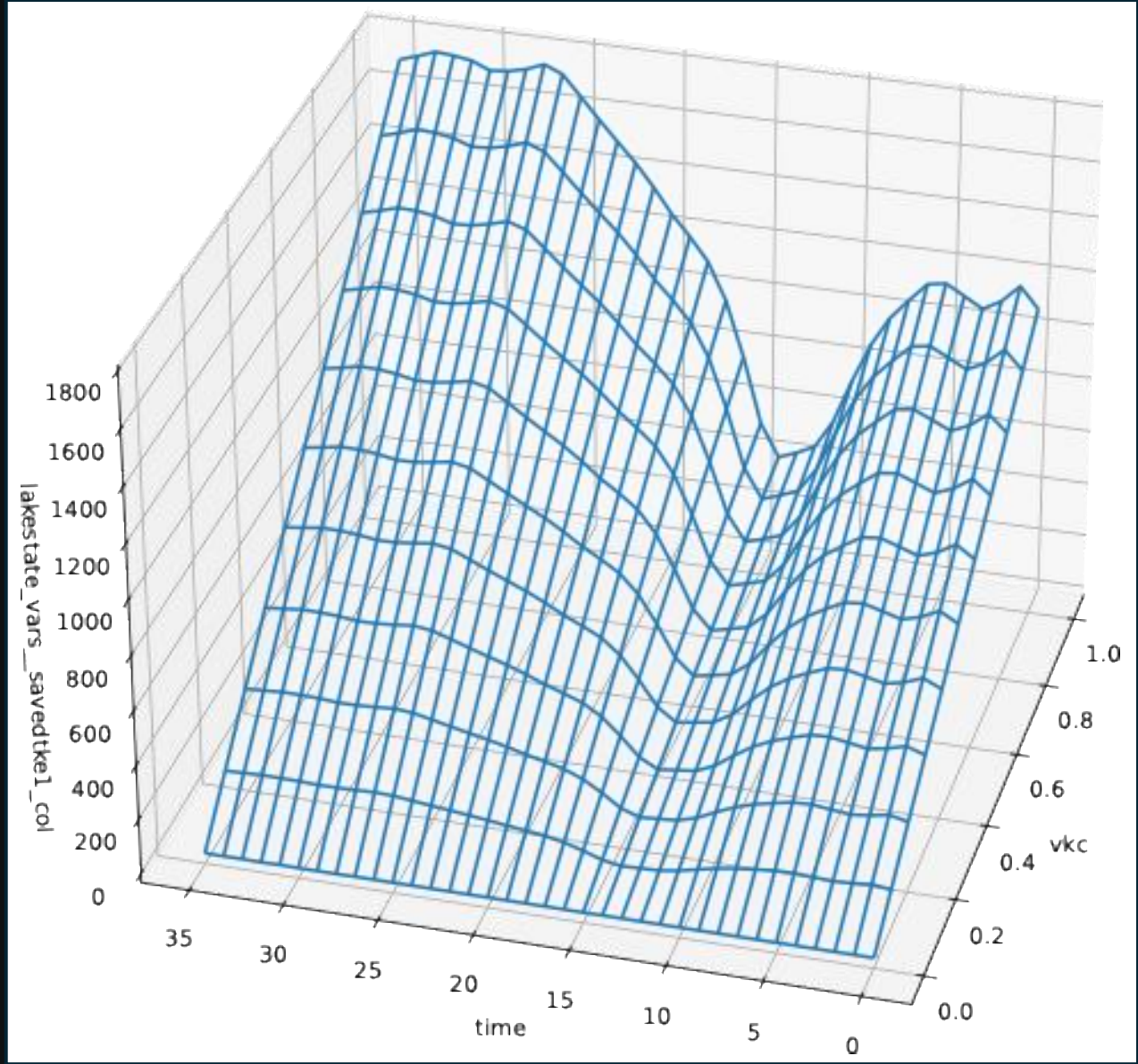
- The goal is to visualize the impact of scalar model constants on model outputs.
- Some challenges are:
  - Most of the 25 model constants we change affect multiple model outputs.
  - The 75 model outputs are multidimensional. The column dimension goes from 0 to 344.
  - Contrary to an initial assumption, some model constants only affect model outputs when certain preconditions are met.
- We created a 3D graph using time as the x-axis, the one model constant we changed as the y-axis, and a single column of a single model output as the z-axis.

Graphing Process

- Output data in the form of a NetCDF file was used.
- From the output data we pulled the values for the model constant *vkc*, which represents the *Von Kármán Constant.*
- Time was interpolated from 0 to 35.
- From the output data we pulled the values for the model output *lakestate_vars__savedtke1_col*–the *eddy conductivity* at the top lake level saved for the next step–at the column numbered 336.
- Using matplotlib, all three dimensions were graphed.

An Example Graph

- This graph shows how the model constant *vkc* affects the model output *lakestate_vars__savedtke1_col* over a period of 35 time steps for the lake column numbered 336.



Part 3: Combinatorial Testing

Why Combinatorial Testing (CT)?

- Ideally, `LakeTemperature` should be validated against all possible combinations of model constant values to ensure that no combination results in erroneous output.
- Exhaustive testing is neither practical nor feasible due to the vast number of all possible combinations.
- Combinatorial testing substantially reduces the number of required test cases while achieving near-exhaustive coverage.
- Assuming most of the bugs are triggered by the interactions of model constants

Background of Combinatorial Testing

- NIST research found most software bugs are caused by the interactions of only a small number of parameters[1].
- By testing all combinations of a certain number of parameters, most software bugs can be found with much fewer test cases.
- NIST's ACTS tool can generate covering arrays from 1-way to 6-way or with mixed coverage[2].

1. *Automated Combinatorial Testing for Software (ACTS)*. NIST Computer Security Division. (2016, July 20). https://csrc.nist.rip/groups/SNS/acts/
2. *User Guide for ACTS*. NIST. (n.d.). https://csrc.nist.gov/csrc/media/Projects/automated-combinatorial-testing-for-software/documents/acts_user_guide_for_basic1.0_and_advanced3.3_versions_nov23.pdf

Application of CT to `LakeTemperature`

- The scalar model constants, which we can control when running `LakeTemperature`, are used as the parameters for the combinatorial test cases.
- 3 scalar model constants are Booleans, 7 are integers, and 15 are floats.
- Booleans can be either 0 or 1.
- For integers, we linearly interpolated over their ranges to get up to 11 values.
- For floats, we linearly interpolated over their ranges to get exactly 11 values.

NetCDF Excerpts for Illustration

```
bool lakepuddling = 0, 1 ;

int nlevsno = 0, 1, 2, 3, 4, 5 ;

int depthcrit = 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35 ;

float thk_bedrock = 2, 2.2, 2.4, 2.6, 2.8, 3, 3.2, 3.4, 3.6, 3.8, 4 ;
```

Test Oracle

- Scientific software verification is challenging due to often lacking a test oracle.
- We implemented checks that validate model outputs, supported by the fault finding analysis plugin.
- Checks include ranges of outputs, signs, enumerated values, finiteness, relationships to other outputs, and certain physical laws and constraints.

Sign Checks Overview

- Check that a certain model output is not negative
- 11 out of 57 checks are of this type.

Example Check

```python
def check_water_snow_equivalent_not_negative(
    test_col_ws_h2osno: npt.NDArray
) -> None:

    assert_not_negative(test_col_ws_h2osno, "col_ws%ws_h2osno")
```

```python
def assert_not_negative(variable: npt.NDArray, name: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all(variable >= 0.0), f"Negative values found in {name}"
```

All Sign Checks

- $\forall t \forall c(\text{ch4\_vars\%grnd\_ch4\_cond\_col}[t, c] \geq 0)$
  - ch4_conductance_check.check_methane_conductance_not_negative
  - Precondition: use_lch4 = 1
- $\forall t \forall c(\text{col\_ws\%h2osno}[t, c] \geq 0)$
  - physical_bounds_check.check_water_snow_equivalent_not_negative
- $\forall t \forall c(\text{col\_ws\%snow\_depth}[t, c] \geq 0)$
  - physical_bounds_check.check_snow_depth_not_negative
- $\forall t \forall j \forall c(\text{col\_wf\%qflx\_snofrz\_lyr}[t, j, c] \geq 0)$
  - physical_bounds_check.check_snow_freeze_rate_not_negative
- $\forall t \forall c(\text{col\_wf\%qflx\_snomelt}[t, c] \geq 0)$
  - physical_bounds_check.check_snow_melt_flux_not_negative

All Sign Checks (Continued)

- $\forall t \forall c(\text{col\_wf\%qflx\_snow\_melt}[t, c] \geq 0)$
  - physical_bounds_check.check_net_snow_melt_not_negative
- $\forall t \forall c(\text{col\_ef\%eflx\_snomelt}[t, c] \geq 0)$
  - physical_bounds_check.check_snow_melt_heat_flux_not_negative
- $\forall t \forall c(\text{lakestate\_vars\%lakeresist\_col}[t, c] \geq 0)$
  - physical_bounds_check.check_lake_water_transport_resistance_not_negative
- $\forall t \forall c(\text{lakestate\_vars\%savedtke1\_col}[t, c] \geq 0)$
  - physical_bounds_check.check_saved_eddy_conductivity_not_negative
- $\forall t \forall c(\text{col\_es\%hc\_soi}[t, c] \geq 0)$
  - physical_bounds_check.check_soil_heat_content_not_negative
- $\forall t \forall c(\text{col\_es\%hc\_soisno}[t, c] \geq 0)$
  - physical_bounds_check.check_combined_heat_content_not_negative

Range Checks Overview

- Check that a certain model output is in [0, 1]
- 2 out of 57 checks are of this type.

```python
def check_surface_absorption_fraction_is_fraction(
    test_lakestate_vars_betaprime_col: npt.NDArray
) -> None:

    assert_is_frac(test_lakestate_vars_betaprime_col, "lakestate_vars%betaprime_col")
```

Example Check

```python
def assert_is_frac(variable: npt.NDArray, name: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all((variable >= 0.0) & (variable <= 1.0)), (
        f"Non-fractional values found in {name}"
    )
```

All Range Checks

- $\forall t \forall j \forall c(\text{lakestate\_vars\%lake\_icefrac\_col}[t, j, c] \in [0, 1])$
  - physical_bounds_check.check_lake_layer_ice_fraction_is_fraction
- $\forall t \forall c(\text{lakestate\_vars\%betaprime\_col}[t, c] \in [0, 1])$
  - physical_bounds_check.check_surface_absorption_fraction_is_fraction

Finiteness Checks Overview

- Check that a certain model output is finite
- 22 out of 57 checks are of this type.

Example Check

```python
def check_lake_temperature_finite(test_col_es_t_lake: npt.NDArray) -> None:
    assert_variable_is_finite(test_col_es_t_lake, "col_es%t_lake")

def assert_variable_is_finite(variable: npt.NDArray, name: str) -> None:
    assert np.all(np.isfinite(variable)), f"Infinite or NaN values found in {name}"
```

All Finiteness Checks

- ∀t∀j∀c(col_es%t_lake[t, j, c] != NaN ∩ col_es%t_lake[t, j, c] != inf)
  - finite_values_check.check_lake_temperature_finite
- ∀t∀j∀c(col_es%t_soisno[t, j, c] != NaN ∩ col_es%t_soisno[t, j, c] != inf)
  - finite_values_check.check_soil_snow_temperature_finite
- ∀t∀p(veg_ef%eflx_gnet[t, p] != NaN ∩ veg_ef%eflx_gnet[t, p] != inf)
  - finite_values_check.check_ground_net_heat_flux_finite
- ∀t∀p(veg_ef%eflx_sh_grnd[t, p] != NaN ∩ veg_ef%eflx_sh_grnd[t, p] != inf)
  - finite_values_check.check_ground_sensible_heat_flux_finite
- ∀t∀p(veg_ef%eflx_sh_tot[t, p] != NaN ∩ veg_ef%eflx_sh_tot[t, p] != inf)
  - finite_values_check.check_total_sensible_heat_flux_finite

All Finiteness Checks (Continued)

- ∀t∀p(veg_ef%eflx_soil_grnd[t, p] != NaN ∩ veg_ef%eflx_soil_grnd[t, p] != inf)
  - finite_values_check.check_ground_heat_flux_finite
- ∀t∀c(col_ef%eflx_snomelt[t, c] != NaN ∩ col_ef%eflx_snomelt[t, c] != inf)
  - finite_values_check.check_snow_melt_heat_flux_finite
- ∀t∀j∀c(col_wf%qflx_snofrz_lyr[t, j, c] != NaN ∩ col_wf%qflx_snofrz_lyr[t, j, c] != inf)
  - finite_values_check.check_snow_freeze_rate_finite
- ∀t∀c(col_wf%qflx_snomelt[t, c] != NaN ∩ col_wf%qflx_snomelt[t, c] != inf)
  - finite_values_check.check_snow_melt_water_flux_finite
- ∀t∀c(col_wf%qflx_snow_melt[t, c] != NaN ∩ col_wf%qflx_snow_melt[t, c] != inf
  - finite_values_check.check_new_snow_melt_rate_finite
- ∀t∀c(col_ws%h2osno[t, c] != NaN ∩ col_ws%h2osno[t, c] != inf)
  - finite_values_check.check_snow_water_equivalent_finite

All Finiteness Checks (Continued)

- ∀t∀c(col_ws%snow_depth[t, c] != NaN ∩ col_ws%snow_depth[t, c] != inf)
  - finite_values_check.check_snow_depth_finite
- ∀t∀c(lakestate_vars%lakeresist_col[t, c] != NaN ∩ lakestate_vars%lakeresist_col[t, c] != inf)
  - finite_values_check.check_lake_transport_resistance_finite
- ∀t∀c(lakestate_vars%savedtke1_col[t, c] != NaN ∩ lakestate_vars%savedtke1_col[t, c] != inf)
  - finite_values_check.check_saved_eddy_conductivity_finite
- ∀t∀c(lakestate_vars%grnd_ch4_cond_col[t, c] != NaN ∩ lakestate_vars%grnd_ch4_cond_col[t, c] != inf)
  - finite_values_check.check_ground_methane_conductance_finite
  - Precondition: use_lch4 = 1
- ∀t∀c(col_ef%errsoi[t, c] != NaN ∩ col_ef%errsoi[t, c] != inf)
  - finite_values_check.check_energy_conservation_residual_finite

All Finiteness Checks (Continued)

- ∀t∀j∀c(col_ef%imelt[t, j, c] != NaN ∩ col_ef%imelt[t, j, c] != inf)
  - finite_values_check.check_snow_layer_flag_finite
- ∀t∀c(col_es%hc_soi[t, c] != NaN ∩ col_es%hc_soi[t, c] != inf)
  - finite_values_check.check_soil_heat_content_finite
- ∀t∀c(col_es%hc_soisno[t, c] != NaN ∩ col_es%hc_soisno[t, c] != inf)
  - finite_values_check.check_combined_heat_content_finite
- ∀t∀c(lakestate_vars%betaprime_col[t, c] != NaN ∩ lakestate_vars%betaprime_col[t, c] != inf)
  - finite_values_check.check_surface_absorption_finite
- ∀t∀j∀c(lakestate_vars%lake_icefrac_col[t, j, c] != NaN ∩ lakestate_vars%lake_icefrac_col[t, j, c] != inf)
  - finite_values_check.check_ice_mass_fraction_finite
- ∀t∀c(lakestate_vars%lake_icethick_col[t, c] != NaN ∩ lakestate_vars%lake_icethick_col[t, c] != inf)
  - finite_values_check.check_ice_thickness_finite

Enumerated Value Check Overview

- Check that a certain model output consists of valid enumerated values
- 1 out of 57 checks are of this type.

Example Check

```python
def check_imelt_uses_valid_enum_values(
    test_col_ef_imelt: npt.NDArray,
) -> None:
    # Values of 2147483647 are uninitialized, and should be masked.
    masked_imelt = np.ma.masked_where(
        test_col_ef_imelt == 2147483647, test_col_ef_imelt
    )
    if NonFiniteValuesHandler.is_all_not_finite(masked_imelt):
        return CheckStatus.SKIPPED
    masked_imelt = NonFiniteValuesHandler.mask_non_finite_values(masked_imelt)

    assert np.all(masked_imelt >= 0) and np.all(masked_imelt <= 2)
```

All Enumerated Value Checks

- $\forall t \forall j \forall c(\text{col\_ef\%imelt}[t, j, c] \in \{0, 1, 2\})$
  - aggregation_and_consistency_check.check_imelt_uses_valid_enum_values

Relationship to Other Outputs Checks Overview

- Check that a certain model output has a certain relationship to other model outputs
- 9 out of 57 checks are of this type.

Example Check

```python
def check_snofrz_lyr_sums_to_snofrz_col(
    test_col_wf_qflx_snofrz_lyr: npt.NDArray,
    test_col_wf_qflx_snofrz: npt.NDArray,
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_wf_qflx_snofrz_lyr,
                                                test_col_wf_qflx_snofrz):
        return CheckStatus.SKIPPED
    test_col_wf_qflx_snofrz_lyr, test_col_wf_qflx_snofrz = (
        NonFiniteValuesHandler.mask_non_finite_values(test_col_wf_qflx_snofrz_lyr,
                                                      test_col_wf_qflx_snofrz))

    tolerance = 1E-10

    lyr_sum = test_col_wf_qflx_snofrz_lyr.sum(axis=1)
    abs_diff = np.abs(test_col_wf_qflx_snofrz - lyr_sum)

    assert np.all(abs_diff <= tolerance), "Sum does not match expected values"
```

All Relationship to Other Outputs Checks

- $\forall t \forall c((\sum_j \text{lakestate\_vars\%lake\_icefrac\_col}[t, j, c] \times \text{col\_pp\%dz\_lake}[t, j, c] \times \text{denh2o} \div \text{denice}) \approx \text{lakestate\_vars\%lake\_icethick\_col}[t, c])$
  - aggregation_and_consistency_check.check_icethick_col_is_sum
- $\forall t \forall c(\Delta(\text{col\_es\%hc\_soisno}[t, c])/\Delta t \approx \int(\sum_p(\text{veg\_ef\%eflx\_gnet}[t, p] + \text{veg\_ef\%eflx\_soil\_grnd}[t, p] + \text{veg\_ef\%eflx\_sh\_grnd}[t, p]))\, dt$, where veg_pp%column
  $c)$
  - key_physical_laws_check.check_heat_contents_close
  - Precondition: $\forall t \forall c(\text{col\_pp\%snl}[t, c] = 0 \cap \text{col\_ws\%h2osno}[t, c] = 0 \cap \text{col\_es\%t\_lake}[t, 0, c] > \text{tfrz} \cap$ lakestate
    $0)$
- $\forall t \forall c((\Delta(\sum_j \text{col\_ws\%h2osoi\_ice}[t, j, c])/\Delta t) \times \text{hfus} \times 10^{-6} \approx \Delta(\text{col\_es\%hc\_soisno}[t, c])/\Delta t)$
  - key_physical_laws_check.check_heat_diff_close
  - Precondition: $(\exists t \exists c(\text{col\_pp\%snl}[t, c] < 0 \cap \text{col\_ws\%h2osno}[t, c] > 0) \cap \forall t \forall c(\text{col\_es\%t\_lake}[t, 0, c] \leq \text{tfrz}) \cap$
    $\text{tfrz})$

All Relationship to Other Outputs Checks (Continued)

- $\forall t \forall c(\sum_j \text{col\_wf\%qflx\_snofrz\_lyr}[t, c] \approx \text{col\_wf\%qflx\_snofrz}[t, c])$
  - aggregation_and_consistency_check.check_snofrz_lyr_sums_to_snofrz_col
- $\forall t \forall c(\text{col\_ef\%eflx\_snomelt}[t, c] = \text{col\_wf\%qflx\_snomelt}[t, c] \times \text{hfus})$
  - key_physical_laws_check.check_energy_flux_consistent_with_latent_heat
  - Precondition: $\exists t \exists c(\text{col\_pp\%snl}[t, c] = 0 \cap \text{col\_ws\%h2osno}[t, c] > 0) \cap \forall t \forall c(\text{col\_es\%t\_lake}[t, 0, c$ tfrz)
- $\forall t \forall c(\text{col\_es\%hc\_soisno}[t, c] \geq \text{col\_es\%hc\_soi}[t, c])$
  - physical_bounds_check.check_combined_heat_content_not_less_than_soil_heat_content
- $\forall t \forall c(\text{lakestate\_vars\%lake\_icefrac\_col}[t, 0, c] = 0 \rightarrow (1/(\text{lakestate\_vars\%lakeresist\_col}[t, c] + \text{lakestat}$ ch4_vars%grnd_ch4_cond_col[t, c])
  - key_physical_laws_check.check_methane_conductance_allowed_without_ice
  - Precondition: use_lch4 = 1

All Relationship to Other Outputs Checks (Continued)

- $\forall t \forall c((col\_pp\%snl[t, c] = 0 \cap col\%ws\_h2osno[t, c] > 0) \cap (col\_pp\%snl[t + dt, c] = 0 \cap col\%ws\_h2o$ $dt, c])$
  - key_physical_laws_check.check_snow_depth_decreases_with_snow_melt_rate
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] = 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0$ tfrz)
- $\forall t \forall c((col\_pp\%snl[t, c] = 0 \cap col\%ws\_h2osno[t, c] > 0) \cap (col\_pp\%snl[t + dt, c] = 0 \cap col\%ws\_h2o$ $dt, c])$
  - key_physical_laws_check.check_snow_depth_decreases_with_snow_melt_rate
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] = 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0$ tfrz)

Physical Laws and Constraints Checks Overview

- Check that a certain model output obeys physical laws and constraints
- 12 out of 57 checks are of this type.

Example Check

```python
def check_temp_around_freezing_where_lake_is_almost_frozen(
    test_col_es_t_lake: npt.NDArray,
    test_lakestate_vars_lake_icefrac_col: npt.NDArray,
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_es_t_lake,
                                                test_lakestate_vars_lake_icefrac_col):
        return CheckStatus.SKIPPED
    test_col_es_t_lake, test_lakestate_vars_lake_icefrac_col = (
        NonFiniteValuesHandler.mask_non_finite_values(test_col_es_t_lake,
                                                      test_lakestate_vars_lake_icefrac_col))

    almost_frozen = (np.isclose(test_lakestate_vars_lake_icefrac_col, 1.0)
                     & (test_lakestate_vars_lake_icefrac_col != 1.0))

    if not np.any(almost_frozen):
        return CheckStatus.SKIPPED

    diff_to_freezing_temp = np.abs(TFRZ - test_col_es_t_lake)
    close_to_freezing_temp = diff_to_freezing_temp <= 1E-3

    assert np.all(~almost_frozen | close_to_freezing_temp), (
        "Columns are almost frozen but not close to the freezing temperature"
    )
```

All Physical Laws and Constraints Checks

- $\forall t \forall c(\text{lakestate\_vars\%lake\_icefrac\_col}[t, 0, c] > 0.1 \rightarrow \text{ch4\_vars\%grnd\_ch4\_cond\_col}[t, c] \approx 0)$
  - ch4_conductance_check.check_methane_conductance_frozen_lake
  - Precondition: use_lch4 = 1
- $\forall t \forall c(\text{col\_ef\%errsoi}[t, c] \approx 0)$
  - key_physical_laws_check.check_errsoi_threshold
  - Precondition: $\forall t \forall c(\text{col\_pp\%snl}[t, c] = 0 \cap \text{col\_ws\%h2osno}[t, c] = 0 \cap \text{col\_es\%t\_lake}[t, 0, c] > \text{tfrz} \cap \text{lakestate\_}$ $0)$
- $\forall t \forall c(\text{col\_pp\%snl}[t, c] < 0 \cap \text{col\_ws\%h2osno}[t, c] > 0 \rightarrow \text{col\_wf\%qflx\_snofrz\_lyr}[t, 0, c] > 0)$
  - key_physical_laws_check.check_surface_snow_freezing_where_snow_present
  - Precondition: $\exists t \exists c(\text{col\_pp\%snl}[t, c] < 0 \cap \text{col\_ws\%h2osno}[t, c] > 0) \cap \forall t \forall c(\text{col\_es\%t\_lake}[t, 0, c] \leq \text{tfrz} \cap \text{col\_}$ $\text{tfrz})$
- $\forall t \forall c(\text{col\_pp\%snl}[t, c] < 0 \cap \text{col\%ws\_h2osno}[t, c] > 0 \rightarrow \text{col\_ef\%imelt}[t, 0, c] = 2)$
  - key_physical_laws_check.check_snow_labeled_freezing_where_snow_present
  - Precondition: $\exists t \exists c(\text{col\_pp\%snl}[t, c] < 0 \cap \text{col\_ws\%h2osno}[t, c] > 0) \cap \forall t \forall c(\text{col\_es\%t\_lake}[t, 0, c] \leq \text{tfrz} \cap \text{col\_}$ $\text{tfrz})$

All Physical Laws and Constraints Checks (Continued)

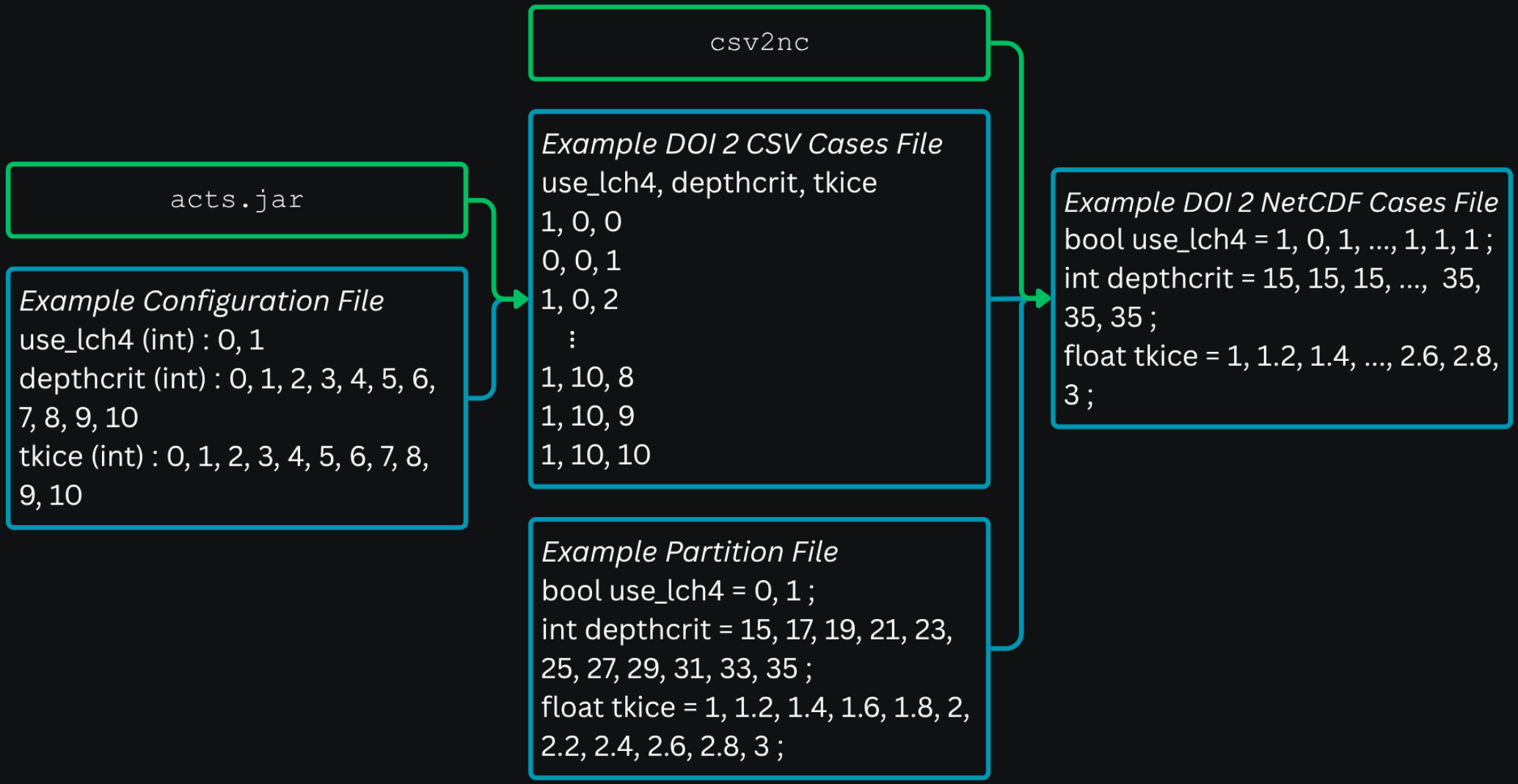
- $\forall t \forall c(col\_pp\%snl[t, c] < 0 \cap col\_ws\%h2osno[t, c] > 0 \rightarrow col\_wf\%qflx\_snomelt[t,c] = 0)$
  - key_physical_laws_check.check_snow_not_melting_where_snow_present
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] < 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0, c] \leq tfrz \cap col\_$ tfrz)
- $\forall t \forall c(\Delta(col\_ws\%h2osno[t, c])/\Delta t \geq 0)$
  - key_physical_laws_check.check_snow_water_not_decreasing
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] < 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0, c] \leq tfrz \cap col\_$ tfrz)
- $\forall t \forall c(\Delta(col\_ws\%snow\_depth[t, c])/\Delta t \geq 0)$
  - key_physical_laws_check.check_snow_depth_not_decreasing
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] < 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0, c] \leq tfrz \cap col\_$ tfrz)
- $\forall t \forall c(col\_pp\%snl[t, 0, c] = 0 \cap col\_ws\%h2osno[t, c] > 0 \cap col\_es\%t\_lake[t, 0, c] > tfrz \rightarrow col\_wf\%qflx\_snomelt[t, c] > 0)$
  - key_physical_laws_check.check_snow_melting_where_snow_water_present
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] = 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t, 0, c] > tfrz)$

All Physical Laws and Constraints Checks (Continued)

- $\forall t \forall c(col\_pp\%snl[t, c] = 0 \cap col\_ws\%h2osno[t, c] > 0 \cap col\_es\%t\_lake[t, 0, c] > tfrz \rightarrow col\_wf\%qflx\_snow\_melt[t, c] > 0)$
  - key_physical_laws_check.check_snow_melted_where_snow_water_present
  - Precondition: $\exists t \exists c(col\_pp\%snl[t, c] = 0 \cap col\_ws\%h2osno[t, c] > 0) \cap \forall t \forall c(col\_es\%t\_lake[t$ tfrz)
- $\neg \exists t \exists c(ch4\_vars\%grnd\_ch4\_cond\_col[t, c] > 0 \cap lakestate\_vars\%lake\_icefrac\_col[t, 0, c] > 0.1)$
  - key_physical_laws_check.check_methane_conductance_gated_by_ice
  - Precondition: $use\_lch4 = 1$
- $\forall t \forall j \forall c(lakestate\_vars\%lake\_icefrac\_col[t, j, c] \approx 1 \cap lakestate\_vars\%lake\_icefrac\_col[t, j, c] \neq 1 \rightarrow 0)$
  - physics_check.check_temp_around_freezing_where_lake_is_almost_frozen
- $\forall t \forall c(lakestate\_vars\%savedtke1\_col[t, c] > 0 \rightarrow lakestate\_vars\%lake\_icefrac\_col[t, 0, c] \approx 0 \cap col\_pp\%snl[t, c] = 0)$
  - physics_check.check_surface_unfrozen_when_tke_present

Workflow for Sample Generation

- The configuration file provides the indices for the number of values each model constant can take.
- ACTS uses these indices to generate covering arrays from 2-way to 5-way and outputs a CSV file for each.
- The partition file provides the values that correspond to the indices in the csv files.
- The Python script (`csv2nc`) uses the partition file's values to create NetCDF files which are run on the model using the NetCDF cases sampling plugin.


csv2nc
acts.jar
Example Configuration File
use_lch4 (int) : 0, 1
depthcrit (int) : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
tkice (int) : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Example DOI 2 CSV Cases File
use_lch4, depthcrit, tkice
1, 0, 0
0, 0, 1
1, 0, 2
⋮
1, 10, 8
1, 10, 9
1, 10, 10
Example Partition File
bool use_lch4 = 0, 1 ;
int depthcrit = 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35 ;
float tkice = 1, 1.2, 1.4, 1.6, 1.8, 2, 2.2, 2.4, 2.6, 2.8, 3 ;
Example DOI 2 NetCDF Cases File
bool use_lch4 = 1, 0, 1, ..., 1, 1, 1 ;
int depthcrit = 15, 15, 15, ..., 35, 35, 35 ;
float tkice = 1, 1.2, 1.4, ..., 2.6, 2.8, 3 ;

Test Suite Sizes and Timings for Cases with All Scalar Model Constants

For ten samples, the mean time it takes to run `elmtest` (for 35 time steps) is 0.622 seconds, with a standard deviation of 0.030. This time is used to estimate how long testing each interaction level would take, assuming tests run sequentially.

*Large test cases haven't been executed yet.

DOI	Case Count	Estimated Execution Time	Actual Execution Time
2	231	0 days, 00:02:24	0 days, 00:02:17
3	4,706	0 days, 00:48:47	0 days, 00:46:50
4	72,964	0 days, 12:36:24	- days, --:--:--*
5	1,009,602	7 days, 06:26:12	- days, --:--:--*

ACTS Case Generation Timings for Cases with All Scalar Model Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.303	0.016
3	4,706	1.638	0.027
4	72,964	201.699	2.937
5	1,009,602	15421.421	171.548

- This represents the amount of time it took to generate the test cases that include all scalar model constants using ACTS.
- The mean and standard deviation were calculated using 7 runs of ACTS for each DOI level.

`csv2nc` Script Timings for Cases with All Scalar Model Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.671	0.030
3	4,706	6.187	0.102
4	72,964	88.892	0.382
5	1,009,602	1155.359	16.407

- This represents the amount of time it took to generate the test cases that include all scalar model constants using `csv2nc`.
- The mean and standard deviation were calculated using 7 runs of `csv2nc` for each DOI level.

Test Suite Sizes and Timings for Cases with Some Model Constants*

The estimated execution times for cases with some constants use the same mean `elmtest` time as the estimated execution times for cases with all scalar model constants.

*Combinatorial cases with some constants exclude the constants originally believed to have no effect on model outputs (following the change detection analysis) in addition to exluding the 3 non-scalar model constants.

**Large test cases haven't been executed yet.

DOI	Case Count	Estimated Execution Time	Actual Execution Time
2	231	0 days, 00:02:24	0 days, 00:02:20
3	3,685	0 days, 00:38:12	0 days, 00:37:28
4	53,170	0 days, 09:11:12	- days, --:--:--**
5	688,290	4 days, 22:55:16	- days, --:--:--**

ACTS Case Generation Timings for Cases with Some Model Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.213	0.006
3	3,685	0.632	0.024
4	53,170	12.641	0.197
5	688,290	1107.923	16.651

- This represents the amount of time it took to generate the test cases that include some model constants using ACTS.
- The mean and standard deviation were calculated using 7 runs of ACTS for each DOI level.

`csv2nc` Script Timings for Cases with Some Model Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.648	0.114
3	3,685	3.226	0.054
4	53,170	41.233	1.146
5	688,290	501.764	2.276

- This represents the amount of time it took to generate the test cases that include some model constants using `csv2nc`.
- The mean and standard deviation were calculated using 7 runs of `csv2nc` for each DOI level.

DOI 2 Results

- Degree of Interaction (DOI) 2 results in 231 test cases when only changing the values of constants known to affect the outputs of `LakeTemperature`.
- 33 runs exited with an error because they used values outside of the valid ranges for some model constants.

33 out of 133 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_combined_heat_content_finite	5	0	95	0
check_combined_heat_content_not_less_than_soil_heat_content	98	0	2	0
check_energy_conservation	15	67	18	0
check_energy_conservation_residual_finite	5	0	95	0
check_errsoi_almost_zero	75	0	25	0
check_freezing_latent_heat	0	100	0	0
check_ground_heat_flux_finite	100	0	0	0
check_ground_methane_conductance_finite	73	0	27	0
check_ground_net_heat_flux_finite	100	0	0	0
check_ground_sensible_heat_flux_finite	100	0	0	0
check_ice_mass_fraction_finite	100	0	0	0
check_ice_thickness_finite	33	0	67	0
check_icethick_col_is_sum	100	0	0	0
check_imelt_uses_valid_enum_values	100	0	0	0
check_lake_layer_ice_fraction_is_fraction	100	0	0	0
check_lake_temperature_finite	5	0	95	0
check_lake_transport_resistance_finite	73	0	27	0
check_lake_water_transport_resistance_not_negative	100	0	0	0
check_melting_latent_heat	0	100	0	0
check_methane_conductance_allowed_without_ice	50	50	0	0
check_methane_conductance_frozen_lake	16	84	0	0
check_methane_conductance_gated_by_ice	16	84	0	0
check_methane_conductance_non_negative	50	50	0	0
check_net_snow_melt_not_negative	100	0	0	0
check_new_snow_melt_rate_finite	100	0	0	0
check_saved_eddy_conductivity_finite	100	0	0	0
check_saved_eddy_conductivity_not_negative	100	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	100	0	0	0
check_snow_depth_finite	100	0	0	0
check_snow_depth_not_negative	100	0	0	0
check_snow_freeze_rate_finite	100	0	0	0
check_snow_freeze_rate_not_negative	100	0	0	0
check_snow_layer_flag_finite	100	0	0	0
check_snow_melt_flux_not_negative	100	0	0	0
check_snow_melt_heat_flux_finite	100	0	0	0
check_snow_melt_heat_flux_not_negative	100	0	0	0
check_snow_melt_water_flux_finite	100	0	0	0
check_snow_water_equivalent_finite	100	0	0	0
check_soil_heat_content_finite	5	0	95	0
check_soil_heat_content_not_negative	93	0	7	0
check_soil_snow_temperature_finite	5	0	95	0
check_surface_absorption_finite	100	0	0	0
check_surface_absorption_fraction_is_fraction	100	0	0	0
check_temp_around_freezing_where_lake_is_almost_frozen	0	100	0	0
check_total_sensible_heat_flux_finite	100	0	0	0
check_water_snow_equivilant_not_negative	100	0	0	0

DOI 2 Results Explanation

- These checks were run using a 35 time step limit. Once we finalize our checks, we will run them using the 1000 time step limit. As `LakeTemperature` runs longer, we are more likely to expose issues with its design and/or implementation.
- Some checks were skipped because they are only valid under certain preconditions. For example, check_methane_conductance_allowed_without_ice cannot run when use_lch4 == 0, when methane is turned off.
- We cannot ensure some of the preconditions are met because they rely on the starting values of some model inputs, which we cannot control.
- Every check makes an assertion about model output data. When that assertion fails, the check fails. This can point to a problem in the design of the check, SPEL, or `LakeTemperature`.
- A check outputs an error when it is unable to run correctly, e.g., if there was a syntax mistake. Errors point to bugs in our code that will be fixed prior to final testing.

DOI 3, 4, 5 Results

- When only changing the values of constants known to affect the outputs of `LakeTemperature`
  - DOI 3 results in 3,516 test cases.
  - DOI 4 results in 50,031 test cases.
  - DOI 5 results in 689,735 test cases.
- We did not run these since running DOI 2 helped us find problems in our checks that we needed to fix before re-running the test cases and getting meaningful results.

Improvements to the Testing Framework

- Our test cases originally had invalid values for constants that could only be used when certain preconditions were met, which caused crashes.
- Some checks were implemented incorrectly, which were fixed.
- For example, tests checking for things other than non-finite values had to ignore non-finite values, otherwise they would fail and output an inaccurate error message.

Findings from Preliminary Testing

- SPEL might be initializing data with NaN values that `LakeTemperature` then never overwrites or `LakeTemperature` might be outputting data with NaN values. Currently .../231 combinatorial cases with all constants have at least one model output with NaN values and .../231 combinatorial cases with some constants have at least one model output with NaN values.
- Largely, `LakeTemperature` output data appeared valid and within physical bounds. However, some cases resulted in physically impossible outputs, likely due to the interactions between model constants.